Faculty of Engineering Department of
Artificial intelligence

Generative Plagiarism Detection System

Senior1 Project – Completed the requirements for obtaining a bachelor's degree in Artificial Intelligence Engineering

Prepared by:

Farah Alkayyem Tala Alkhateeb

Supervised by:

Dr. Riad Sonbol

Academic Year: 2026 – 2027

كلية هندسة الذكاء الصنعي

نظام كشف الانتحال المولّد

مشروع تخرج1 – قدم استكمالاً لمتطلبات الحصول على درجة البكالوريوس في هندسة المعلوماتية
هندسة الذكاء الصنعي

إعداد

فرح ربيع القيم تالة محمد فارس الخطيب

إشراف

الدكتور رياض سنبل

السنة الدراسية: 2026 - 2027

Supervisor Certification

I certify that the preparation of this project entitled Generative Plagiarism Detection System, prepared by Farah Alkayyem and Tala Alkhateeb was made under our supervision at Faculty of Artificial Intelligence Engineering in partial Fulfillment of the Requirements for the Degree of Bachelor of Artificial Intelligence engineering.

Name: Dr. Riad Sonbol Signature…… Date:……

Abstract

The increasing accessibility of large language models has given rise to a new form of academic misconduct known as generative plagiarism, in which AI-generated text is submitted as original work despite being semantically derived from existing academic sources. Traditional plagiarism detection tools are largely ineffective against this form of dishonesty, as the generated text shares no verbatim overlap with the underlying sources.

This report presents a Generative Plagiarism Detection system – a full-stack web platform designed to help both students and instructors identify potential AI-generated plagiarism in academic submissions. The system provides a structured environment for uploading documents, managing submissions, and receiving detailed plagiarism analysis reports. The platform includes a standalone generative text detection component where users can independently submit text or documents to obtain an immediate AI probability score. The actual detection logic is handled by a dedicated AI microservice, keeping the main application focused on user experience and workflow management.

الملخص

أدى تزايد إمكانية الوصول إلى النماذج اللغوية الضخمة إلى ظهور شكل جديد من أشكال سوء السلوك الأكاديمي يعرف بـ "الانتحال المولد", وفيه يتم تقديم نصوص مولدة بواسطة الذكاء الاصطناعي على أنها عمل أصلي, رغم كونها مشتقة دلالياً من مصادر أكاديمية قائمة.
وتعد أدوات كشف الانتحال التقليدية غير فعالة إلى حد كبير في مواجهة هذا النوع من عدم الأمانة الأكاديمية, نظراً لأن النصوص المولدة لا تشترك في أي تطابق لفظي مباشر مع المصادر الأساسية.

يقدم هذا التقرير نظاماً لكشف الانتحال المولد – وهو منصة ويب متكاملة صممت لمساعدة كل من الطلاب والمحاضرين في تحديد احتمالات الانتحال المولد آلياً في الأبحاث الأكاديمية المقدمة.
يوفر النظام بيئة مهيكلة لرفع المستندات, وإدارة الطلبات, واستلام تقارير تحليلية مفصلة عن الانتحال. كما يتضمن النظام مكوناً مستقلاً لكشف النصوص المولدة يتيح للمستخدمين تقديم المستندات مباشرة للحصول على درجة تقييم فورية تبين نسبة احتمال استخدام الذكاء الاصطناعي. كما تتم معالجة منطق الكشف الفعلي من خلال خدمة برمجية مصغرة مخصصة للذكاء الاصطناعي, مما يتيح للتطبيق الرئيسي التركيز على تجربة المستخدم وإدارة سير العمل.

Table of Contents

Abstract 4
الملخص 5
Table of Contents 6
List of Tables 8
Chapter 1: Introduction 10
1.1 Introduction 10
1.2 Problem Statement 10
1.3 Project Objectives 11
1.4 Proposed System 11
1.5 Report Organization 12
1.6 Summary 13
Chapter 2: Fundamental Concepts and Literature Review 15
2.1 Introduction 15
2.2 Fundamental Concepts 15
2.3 Literature Review of Existing Systems 18
Turnitin 19
iThenticate 20
Copyleaks 21
Originality.AI 22
GPTZero 23
Pangram Labs 24
2.5 Summary 27
Chapter 3: AI Services 29
3.1 Introduction 29
3.2 Key Terminology 29
3.2.1 Shared Task and Evaluation Tools 29
3.2.2 Retrieval Models and Tools 30
3.2.3 Plagiarism Detection Concepts 31
3.2.4 AI-Generated Text Detection Concepts 32
3.3 Literature Review 34
3.3.1 PAN 2025 Generative Plagiarism Detection Approaches 34
3.3.2 PAN 2026 Generative Plagiarism Detection: Comparison of Team Solutions 38

3.3.3 AI-Generated Text Detection 42
References 45

List of Tables

Table 1: Comparative Analysis of Similar Systems and the Target System 26
Table 2: literature study for Generative Plagiarism Detection 36
Table 3: PAN26 Shared task teams solutions comparison 39
Table 4: PAN26 Shared task results 40
Table 5: winning team in PAN26 VS us..... 41
Table 6: Literature study for AI-Generated Text Detection 42
Table 7: Literature study results 43

Chapter 1: Introduction

Chapter 1: Introduction

1.1 Introduction

This chapter introduces the core elements of the **Generative Plagiarism Detection** system, laying the foundation for understanding the challenges of academic integrity in the age of AI, It outlines the project goals, including full source-matching analysis and standalone AI text scoring, the proposed microservices-based architecture, the organization of the report, and a brief summary of key points.

1.2 Problem Statement

Academic institutions lack simple software to manage AI plagiarism checking for many students at once. Current tools only look for exact matching words, so they cannot catch text generated by AI that rewrites source ideas in new words. Besides finding the plagiarism itself, universities need a practical platform that handles document uploading, background processing, subscription limits, and clear result reports for both full source-checking and quick AI score checking.

The specific problems this project addresses are:

- Lack of Integrated Workflows: Most existing tools focus on only one feature, and very few systems combine full source-matching reports with a separate, quick tool where users can upload a document to get an instant AI likelihood score.

- Slow Processing Times: Analyzing documents takes a long time, so the system must process files in the background without freezing the site or crashing web pages.

- Hard-to-Update Code: AI detection models change quickly. If the AI code is mixed directly into the main website code, updating it is hard and breaks things. It must be

kept separate as its own service.

1.3 Project Objectives

The objectives of this project are:

1. Design and develop a full-stack web application that supports document upload, submission management, full source-matching analysis, and a dedicated generative text detection tool for instant AI probability scoring.

2. Implement asynchronous processing using Celery so that document analysis and text scoring does not block the user interface.

3. Integrate the web platform with a separately developed AI microservice via a well-defined HTTP API, maintaining a clean separation of concerns.

4. Develop an optimized AI processing pipeline to handle model inference efficiently, ensuring relatively rapid generation of analysis results and detection scores.

5. Implement plan-based access control to deliver specific features and workflows according to the user's subscription level.

6. Present analysis results in a clear, structured format that allows users to understand both the instant AI score and the degree and origin of detected plagiarism.

7. Follow sound software engineering practices throughout: modular design, documented APIs, version-controlled development, and systematic testing.

1.4 Proposed System

The proposed system consists of three integrated components:

- Web Application Platform — the primary deliverable of this project. Built with Django (backend), React (frontend), SQL-Server (database), and Celery with RabbitMQ (task queue). It handles all user-facing functionality: authentication and role management, document and source upload, submission workflows, asynchronous analysis job queuing, result storage, and result display.

- Plagiarism Analysis Microservice — a FastAPI service responsible for source matching plagiarism detection logic. The service processes suspicious documents against a set of source documents to generate a plagiarism score and a ranked list of matched sources with similarity percentages.

- Generative Text Detection Microservice: A dedicated FastAPI service responsible for AI-generated text analysis. It evaluates uploaded documents to generate an instant score indicating the probability of AI involvement.

The web platform treats both AI microservices as external dependencies, integrated via HTTP calls triggered by Celery background tasks.

The two components communicate over an HTTP API. This separation allows each component to be developed, tested, and updated independently, and allows the AI service to be replaced or upgraded without modifying the web application.

1.5 Report Organization

The remainder of this report is structured as follows:

- Chapter 1 — Introduction
- Chapter 2 — Fundamental Concepts and Literature Review
- Chapter 3 — Project Management
- Chapter 4 — Requirements and System Analysis
- Chapter 5 — System Design
- Chapter 6 — AI Service Experiments and Evaluation
- Chapter 7 — Practical Implementation
- Chapter 8 — Report Overview

1.6 Summary

This chapter has introduced the motivation and objectives of the Generative Plagiarism Detection project. The system addresses a real and growing need in academic institutions by providing a structured, asynchronous, and extensible web platform for managing AI-generated plagiarism detection and AI text scoring. The AI analysis logic is cleanly encapsulated in dedicated microservices, keeping the main application focused on workflow and user experience. The following chapters present the technical and managerial foundations of the project in detail.

Chapter 2: Fundamental Concepts and Literature Review

Chapter 2: Fundamental Concepts and Literature Review

2.1 Introduction

This chapter provides a foundation for understanding the Generative Plagiarism Detection system by exploring the fundamental concepts and reviewing related studies. It examines essential terminology, and the principles behind the technologies used in the system. The system answers two complementary questions about a suspicious document: which source papers it was derived from (source retrieval), and whether it was written by a machine (AI-generated text detection). The aim is to present an overview of AI applications in source retrieval, AI-generated text detection and document analysis, along with insights from existing detection solutions to guide and support the development of the proposed system.

2.2 Fundamental Concepts

This section explains the core concepts, terms, and theoretical principles relevant to the system, including:

- **Web Application Development (Django & React):** An overview of full-stack web development and how modern frameworks enable structured, maintainable, and scalable applications.

- **Asynchronous Task Processing (Celery & RabbitMQ):** An explanation of how long-running operations — such as document analysis — are handled outside the HTTP request-response cycle to preserve application responsiveness. Celery is a distributed task queue for Python that allows the Django application to delegate time consuming jobs to background worker processes. RabbitMQ serves as the message broker, relaying task messages between the application and the workers.

- **Microservice Architecture and API Integration:** discussion of how decomposing an application into independently deployable services improves maintainability and

flexibility. In this project, the AI detection logic is encapsulated in a separate FastAPI microservice, while the main web platform handles user-facing concerns. The two communicate via a REST API — a widely adopted architectural style for inter-service communication based on standard HTTP methods and JSON data exchange. The microservice exposes two independent analysis endpoints, each with its own versioned contract, and it returns its results to the web platform through callbacks.

- **Object Storage (MinIO):** An introduction to object storage and its role in managing unstructured data such as uploaded documents and source files.

- **Generative Plagiarism:** A discussion of the specific form of academic dishonesty this system is designed to detect. Generative plagiarism occurs when a language model synthesizes a new document from existing academic papers, producing text that is semantically derived from the sources but shares no verbatim overlap with them. This makes conventional string-matching and fingerprinting detection tools fundamentally ineffective, representing a growing challenge for academic institutions as large language models become more accessible.

- **Source Retrieval:** An explanation of the detection strategy adopted in this project. Rather than comparing text directly, source retrieval treats detection as an information retrieval task: given a suspicious document, the system identifies and ranks the original academic papers most likely used to generate it. Key concepts include document embeddings, dense and sparse retrieval, hybrid fusion, and similarity scoring – all of which underpin the AI microservice component of the system.

- **AI-Generated Text Detection:** An explanation of how the system estimates the proportion of a document likely to have been generated by a language model. Detection approaches use supervised classifiers trained on human-written and machine-generated text or zero-shot methods that analyze language-model statistics

(such as Binoculars).

- **Quantified Analysis Results:** The system presents two separate types of percentages for each submitted document. For source analysis, it reports an overall source-match percentage representing the portion of the document matched to the available source collection (complemented by the remaining original-content percentage), together with a ranked list of retrieved sources and a match percentage for each source.

  For AI-generated-text detection, it reports the estimated percentage of the document likely to be AI-generated, these values describe the system's analysis and are not proof of plagiarism or AI use.

- **Separate Source-Match and AI-Detection Results**: The source-retrieval and AI-detection pipelines analyze the same submitted document, in parallel, but answer different questions. Source retrieval identifies and ranks documents in the available collection that are similar in meaning or wording to the submitted document, and reports the overall source-match percentage and the match percentage for each retrieved source. AI detection estimates the percentage of the submitted text that may have been generated by an AI system.

  The interface presents these results side by side, without combining them into a single verdict or automatically classifying the document as misconduct. A high source-match percentage indicates substantial similarity to one or more retrieved sources and is reviewed alongside the listed sources; a high AI-generated-text percentage indicates that the detector considers a larger portion of the text likely to be AI-generated.

  Either result may require human review, and neither percentage alone establishes intent or misconduct. The two pipelines are tracked separately so that a failure or delay in one does not prevent the other from returning its result.

- **Limitations and Responsible Interpretation:** Source-match percentages and AI-generated-text percentages are estimates whose reliability depends on the available source collection, the analysis methods, the text language, and the document characteristics. Source similarity does not by itself establish plagiarism, and an AI-detection score does not prove that a passage was written by AI.

  AI detectors can produce false positives and false negatives, can be biased against non-native English writers, and lose accuracy on paraphrased or mixed human-and-AI text. The system therefore presents the two measurements as separate analytical indicators, displays an explicit disclaimer next to every result, and avoids using them as an automatic accusation or final decision.

- **Subscription Plans and Usage Quotas:** An introduction to usage-based access control, in which each subscription plan defines the number of monthly checks, the supported file formats, and whether AI detection is enabled together with a separate monthly AI-check quota, so that computationally expensive analysis is metered per user.

- **Secure Service-to-Service Callbacks:** An explanation of how the web platform authenticates the results sent back by the AI microservice. A shared secret (an API key or HMAC signature) is sent in a request header and verified by the platform, preventing third parties from posting forged analysis results to the callback endpoint.

2.3 Literature Review of Existing Systems

This section analyzes and evaluates existing plagiarism detection platforms in relation to the proposed system. The analysis focuses on how each platform handles document submission, user roles, analysis workflows, and result presentation, the core software concerns of this project. Because the proposed system also detects AI-generated text, the review pays particular attention to how each platform presents AI-detection results and whether it links them to the source documents. Six widely used platforms are reviewed:

Turnitin

Turnitin is the most widely adopted academic plagiarism detection platform globally. It compares submitted documents against a large reference database. In 2023, Turnitin introduced an AI writing detection module (Turnitin Originality) capable of identifying content generated by large language models such as ChatGPT.

Advantages:

1. Largest reference database in the industry, 1.9 billion student papers, archived web content, and premium journal subscriptions.

2. Deep LMS(Learning Management System) integration via with Canvas, Moodle, Blackboard, Microsoft Teams, and others.

3. Comprehensive toolset: similarity detection, AI writing detection, inline grading, and peer review.

4. Supports 176 languages for similarity detection; AI detection currently strongest in English.

5. Detailed similarity report with color-coded match groups and source links.

Disadvantages:

1. Expensive institutional subscription model, pricing is not publicly disclosed and varies significantly depending on the institution.

2. AI writing detection is a paid add-on, not included by default.

3. AI detection identifies AI-generated text in general but cannot identify which specific source papers were used to generate it.

4. False positive concerns: some genuine student work has been incorrectly flagged as AI-generated.

5. No open-source or self-hosted deployment option; all data is processed on Turnitin's servers.

iThenticate

iThenticate, also operated by Turnitin, is designed for researchers, publishers, and professional authors rather than students. It checks manuscripts, grant proposals, theses and dissertations against Trunitin's academic content database and web sources.

Advantages:

1. Targeted at academic publishing workflows, integrates with major Manuscript Tracking Systems (ScholarOne, Editorial Manager).

2. Private submitted-works repository allows publishers to detect collusion or duplicate submissions across their own archive.

3. AI writing detection available as add-on, including AI paraphrasing detection.

4. REST API available for integration into publishing pipelines.

5. Supports multiple languages and file formats including PDF and DOCX.

Disadvantages:

1. No student-facing submission workflow, not designed for classroom or course management use.

2. AI detection identifies AI-generated text generally but cannot attribute it to specific source papers.

3. High cost, minimum pricing makes it inaccessible for individual researchers or small institutions.

4. AI detection is a paid add-on, not included in the base product.

Copyleaks

Copyleaks is a cloud-based content integrity platform offering both traditional plagiarism detection and AI-generated content detection. It supports detection across 30+ languages and covers content generated by leading LLMs including ChatGPT, Gemini, DeepSeek, and Claude. Independent third-party studies have consistently ranked Copyleaks among the most accurate AI detection tools available.

Advantages:

1. Combined plagiarism and AI detection in a single platform, covering 30+ languages.

2. Detects AI-generated and AI-paraphrased content from all major LLMs.

3. REST API and LMS integration for seamless workflow embedding.

4. Strong security and compliance certifications.

5. Code plagiarism detection module for programming assignments.

Disadvantages:

1. Credit-based pricing can be unpredictable and costly at high volume.

2. AI detection and plagiarism detection were historically separate modules, integration is still maturing.

3. Accuracy of AI detection can fluctuate, some independent tests have shown significant variability.

4. No source-level retrieval, detects that a document is AI-generated but cannot identify which papers it was derived from.

Originality.AI

Originality.AI is a web-based platform combining AI-generated content detection with plagiarism checking, primarily targeting content publishers, agencies, educators, and SEO professionals. In May 2025, it released Plagiarism Checker Version 2, featuring improved paraphrase detection accuracy and broader web coverage.

Advantages:

1. Consistently ranked among the most accurate AI detection tools in independent studies.

2. Combined AI detection and plagiarism checking in one dashboard.

3. Pay-as-you-go pricing model is cost-effective for variable usage.

4. API access available for integration into content workflows and educational platforms.

5. Team management features with role assignment (Admin, Manager, User) suited for organizational use.

Disadvantages:

1. High false positive rate on human-written text.

2. Plagiarism detection checks primarily against web content via Google; not connected to a dedicated academic journal database comparable to Turnitin or iThenticate.

3. No dedicated student submission or course management workflow.

4. No detection of which source papers were used to generate AI text, only flags the presence of AI-generated content.

GPTZero

GPTZero is a commercial AI-detection platform designed mainly for educators and students. It estimates the likelihood that a text was machine-generated by combining perplexity and burstiness measurements with a trained classifier, and it reports results at the sentence level, with an explicit “mixed” category for documents that combine human and AI-written text.

Advantages:

1. Sentence-level highlighting together with a document-level classification (human, mixed, or AI), which makes results easy to interpret.

2. Explicit “mixed” category that reflects real-world use, where documents are partly written and partly generated.

3. Education-oriented workflow with a free tier, which makes it easy for instructors and students to adopt.

4. Perplexity and burstiness features give a degree of explainability to the score.

Disadvantages:

1. Variable accuracy on heavily paraphrased or blended (human + AI) text.

2. Focuses on whether a text is AI-generated and does not identify which academic source papers it was derived from.

3. Closed commercial service with no self-hosted deployment option; all documents are processed on the vendor's servers.

4. Scores are probabilistic and can produce false positives on human-written text.

Pangram Labs

Pangram Labs is a commercial AI-detection platform that classifies text as human-written, AI-assisted, or AI-generated and returns a 0–100 AI score, with color-coded passage highlighting and a confidence value for each passage. Its detector is trained with adversarial and hard-negative examples to keep false positives low, and plagiarism checking with links to sources is offered on its paid plans.

Advantages:

1. Passage-level highlighting with per-passage confidence and a three-way classification (human, AI-assisted, AI-generated).

2. Very low false-positive rate reported by the vendor (about 1 in 24,000 for its latest model), supported by hard-negative and adversarial training data.

3. API with segment-level output and batch upload of PDF, DOCX and RTF files, including OCR for scanned documents.

4. Browser extensions and integrations with Google Docs and learning management systems.

Disadvantages:

1. Closed commercial service with no self-hosted deployment option; all documents are processed on the vendor's servers.

2. Accuracy drops on edited or mixed text, where independent reviews found that some AI-edited documents were classified as human-written.

3. Does not attribute a generated text to the specific academic papers it was derived from; plagiarism checking is a separate, paid-plan feature.

4. Usage is limited by plan, for example a daily word cap on the free tier.

In conclusion, the six reviewed platforms provide solid foundations for traditional plagiarism detection and, increasingly, AI-generated content detection. **Turnitin**, **Copyleaks**, and **Originality.AI** on paid plans, **GPTZero** and **Pangram Labs** offer both capabilities, while **GPTZero** and **Pangram Labs** contribute useful sentence-level and passage-level presentations of AI-detection results. However, none of them addresses generative plagiarism

through semantic source retrieval, and none identifies which specific academic papers a submitted document was derived from while also reporting how much of it is AI-generated.

Additionally, none offer an open-architecture, self-hosted platform that cleanly separates detection logic from the user-facing application.

The proposed system fills these gaps by combining a structured web platform with a dedicated AI microservice that runs two independent pipelines in parallel on every submitted document: source retrieval, which returns a ranked list of the academic papers the document is most similar to, with a match percentage for each source, and AI-generated text detection, which returns the estimated percentage of the text that is AI-generated. The two sets of results are presented side by side, so that the instructor can review each one separately, and they are presented as evidence for human review rather than as proof of misconduct. Both pipelines are independently maintainable and extensible.

Table 1: Comparative Analysis of Similar Systems and the Target System

Feature	Turnitin	iThenticate	Copyleaks	Originality.AI	GPTZero	Pangram Labs	Our System
Document Upload & Submission Management	✓	✓	✓	✓	✓	✓	✓
Similarity / Plagiarism Detection	✓	✓	✓	✓	✓	✓	✓
AI-Generated Content Detection	✓ [1]	✓ [1]	✓	✓	✓	✓	✓
Segment-Level AI Highlighting	✓	✓	✓	✓	✓	✓	✓
Generative Plagiarism / Source Retrieval	✗	✗	✗	✗	✗	✗	✓

Ranked Source Attribution	✗	✗	✗	✗	✗	✗	✓
AI Percentage with Ranked Source Matches	✗	✗	✗	✗	✗	✗	✓
Asynchronous Background Processing	✓	✓	✓	✗	✓	✓	✓
Detailed Match Report with Source Links	✓	✓	✓	✓	✓	✓	✓
Open Source	✗	✗	✗	✗	✗	✗	✓

([1]) Partial support: AI detection available but does not identify specific source papers.

2.5 Summary

This chapter introduced the core technologies underpinning the system, defined generative plagiarism as the specific form of misconduct the system targets, and explained how AI-generated text detection complements source retrieval by reporting a separate AI-generated percentage alongside ranked, per-source match percentages, together with the need to interpret these estimates responsibly. It then reviewed six existing detection platforms (Turnitin, iThenticate, Copyleaks, Originality.AI, GPTZero and Pangram Labs), none of which links AI detection to the specific source papers a text was derived from.

Chapter 3: AI Services

Chapter 3: AI Services

3.1 Introduction

This chapter presents the development of the two AI microservices powering the Generative Plagiarism Detection system: the Plagiarism Analysis Microservice, responsible for source-matching and retrieval, and the Generative Text Detection Microservice, responsible for standalone AI probability scoring. It reviews referential studies and related work that informed the design and methodology of both components. As well as detailing the implementation of both services, along with the experimental pipelines and evaluations conducted to refine their performance.

3.2 Key Terminology

3.2.1 Shared Task and Evaluation Tools

Term	Full Name	Description
PAN	—	An international series of shared tasks focused on authorship analysis, plagiarism detection, and related natural language processing problems.
TIRA	TIRA Integrated Research Architecture	A web-based evaluation platform used in shared tasks including PAN. It allows participants to submit software that is executed against held-out test datasets in a controlled environment.
TREC Format	Text REtrieval Conference Format	A standard output format for information retrieval system results, listing query ID, document ID, rank, and score. Used by evaluation tools including ir_measures.

ir_measures	—	A Python library for computing standard information retrieval evaluation metrics including nDCG, RR, Recall@k, etc
PyTerrier	—	An open-source Python framework for information retrieval experimentation built on the Terrier search engine. Used in this project to implement and evaluate the BM25 baseline.

3.2.2 Retrieval Models and Tools

Term	Full Name	Description
BM25	Best Match 25	A classical sparse retrieval algorithm based on term frequency and inverse document frequency. It scores documents by how often query terms appear in them, weighted by how rare those terms are across the corpus.
TF-IDF	Term Frequency–Inverse Document Frequency	A word weighting method. A word scores higher when it appears often in a document but is rare in the rest of the collection.
E5	Embeddings from bidirectional Encoder representations	A family of sentence transformer models trained specifically for text retrieval tasks
BGE-M3	BAAI General Embedding — Multi-lingual, Multi-functionality, multi-granularity	A long-context embedding model developed by the Beijing Academy of Artificial Intelligence (BAAI). It supports up to 8192 tokens per input, allowing full documents to be encoded without chunking.
GTE	General Text Embeddings	An embedding model family from Alibaba, used by team JLP.

Bi-Encoder	—	A model architecture that encodes the query and document independently into separate vectors and computes similarity between them. Used for efficient first-stage retrieval.
Cross-Encoder	—	A reranking model that takes a query-document pair as joint input and outputs a relevance score. Unlike bi-encoders, cross-encoders attend to both texts simultaneously, producing more accurate but slower scoring.
Cosine Similarity	—	A metric that measures the angle between two vectors in a high-dimensional space. A value of 1 indicates identical direction (maximum similarity) and 0 indicates orthogonality (no similarity).
FAISS	Facebook AI Similarity Search	An open-source library developed by Meta for efficient similarity search over large collections of dense vectors. It supports exact and approximate nearest neighbor search using inner product or L2 distance.
Qdrant	—	An open-source vector database designed for storing and searching dense vector embeddings at scale. Used in the AI microservice to store encoded source document chunks and perform similarity search.

3.2.3 Plagiarism Detection Concepts

Term	Full Name	Description
Sparse Retrieval	—	Search based on exact word matches, such as BM25 or TF-IDF. It is fast and good with technical terms, but misses paraphrases.

Dense Retrieval	—	Search based on embeddings, where texts are turned into vectors and compared by meaning. It can match paraphrased text, but may miss exact terms.
Hybrid Retrieval	—	Combining sparse and dense retrieval so that each covers the weaknesses of the other.
Max-Pooling	—	Keeping only the highest score among a document's chunks as the score for the whole document.
Reranking	—	A second step that re-orders the top results from the first retrieval stage with a more accurate (and slower) model.
Score Fusion	—	Combining the scores or rankings from several retrieval methods into one final ranking, for example with a weighted sum of scores.
RRF	Reciprocal Rank Fusion	A fusion method that uses only rank positions. It avoids comparing scores from different methods.
CombSUM	—	A fusion method that sums the scores a document receives across several result lists. Can be used to combine sentence-level results into a document ranking.

3.2.4 AI-Generated Text Detection Concepts

Term	Full Name	Description
MGT Detection	Machine-Generated Text Detection	Deciding whether a text was written by a human or by an AI.

Generator	—	The LLM that produced a machine-written text. Detectors often fail on generators they were not trained on.
LoRA	Low-Rank Adaptation	A cheap fine-tuning method that trains only small added matrices and keeps the main model frozen.
QLoRA	Quantized LoRA	LoRA on a model compressed to 4-bit numbers, which lets large models train on a single GPU.
Ensemble	—	Combining the predictions of several models to get a more stable result.
Data Augmentation	—	Adding changed versions of training texts so the model learns to handle them. Here it usually means obfuscated AI text.
Obfuscation	—	Changing AI text so it looks human, for example by paraphrasing, back-translation, or style mimicry.
Homoglyph Attack	—	Swapping characters for look-alike Unicode characters, or adding invisible characters, to fool detectors.
Stylometry	—	Measuring writing style features such as sentence length and word variety.
Perplexity	—	A measure of how predictable a text is to a language model. AI text tends to be more predictable.

Calibration	—	Making a model's scores match real probabilities, so 0.9 means the model is right about 90% of the time.
ID / OOD	In-Distribution / Out-of-Distribution	ID data resembles the training data. OOD data comes from new generators, domains, or attacks. Generalization to OOD data is the main weakness of current detectors.

3.3 Literature Review

This section reviews the earlier work behind our project, in three parts. The first covers the approaches submitted to the PAN 2025 Generative Plagiarism Detection shared task, which informed how we built our plagiarism detection system. The second comes after the PAN 2026 results were released. It compares the solutions of the participating teams with our own and shows where our choices were similar or different. The third covers the new component we added, AI detection. It reviews methods for telling human text from machine text, with a focus on approaches that suit academic papers and stay lightweight. Together, these parts show how our work follows current research and where it adds something new.

3.3.1 PAN 2025 Generative Plagiarism Detection Approaches

The PAN 2025 task introduced a new benchmark for detecting generative plagiarism at the passage level: given a suspicious document generated by a large language model from an academic source paper, participating systems must identify and align the plagiarized spans with their original source passages. The reviewed works represent the state of the art on this task and directly inform the retrieval and detection strategy adopted in the AI microservice component of the proposed system.

Mo et al. (2025) — Multi-Feature Fusion with Graph-Based Merging [1]

The winning team at PAN 2025 mixed two kinds of signals: meaning (using GloVe word embeddings) and shared words. They used spaCy to split the text, WordNet to reduce words

to their base form, and NLTK to remove common stop words. The system scored text segments on both signals to find possible copied spans. It then merged them with a graph-based method and soft-NMS, which gives clean spans that do not overlap. A final check with several thresholds removed weak candidates. The system reached a PlagDet score of 0.584, a recall of 0.727, and a granularity of 1.0, so each detected span matched exactly one source. Its main weakness is low precision (0.487), because it searched widely for similar meaning. Its preset thresholds may also need retuning for data outside PAN 2025.

Su et al. (2025) — Hierarchical Multi-Model Vector Retrieval [2]

Su et al. built a step-by-step matching pipeline that combines three ways of comparing text: Sentence-BERT and MPNet embeddings for meaning, and TF-IDF for shared words. FAISS quickly finds candidate spans, and cosine similarity across the three methods then re-ranks them. A BERT classifier double-checks the unclear cases, and a block merging step joins neighboring detections. The system scored a PlagDet of 0.496, with a precision of 0.600 and a recall of 0.578. The BERT check helped keep precision higher by removing false matches. Its granularity was 1.275, which means some copied passages were split into several pieces. Its main limit is that recall depends on the small set of top candidates FAISS returns, so the system has to trade speed against recall, as the authors admit.

Luo et al. (2025) — Two-Stage TF-IDF/Jaccard Filtering with BERT Classification [3]

Luo et al. built a two-stage pipeline that keeps computation low. In the first stage, TF-IDF cosine similarity and character 3-gram Jaccard filtering quickly cut the search down to the sentence pairs with the most shared words. In the second stage, a fine-tuned BERT classifier decides which of the remaining pairs are plagiarism. The system is efficient and has reasonable precision (0.582), but its recall is very low (0.177). The first stage throws away many copied spans that share few words with their sources, and this is exactly the kind of copying that generative plagiarism produces. The BERT classifier was also trained on only 10,000 sentence pairs out of about 2.4 million possible ones, which probably limited how well it handles different rewriting styles.

Tang et al. (2025) — E5 Sentence Embeddings with FAISS Retrieval [4]

Tang et al. built a retrieval pipeline around the e5-base-v2 sentence transformer. It turns both the suspicious and source documents into dense vectors, and FAISS approximate nearest-neighbor search finds the candidates. Before encoding, a sliding window cuts documents into overlapping chunks, so copied spans that cross sentence boundaries are still caught. On validation data the system did well (recall 0.73, precision 0.70, F1 0.71), but precision fell to 0.33 on the test set. This suggests the system was tuned too closely to the validation thresholds. The authors note that adding a cross-encoder reranking stage could improve precision a lot.

Greiner-Petter et al. (2025) — PAN 2025 Task Overview and Official Baselines [5]

The task overview paper by Greiner-Petter et al. sets three baselines that use large language models for semantic similarity (Linq-Embed-Mistral, Qwen, and Llama), plus a word n-gram baseline from the PAN 2012 benchmark. The Linq-Embed-Mistral baseline did better than most team submissions, with a PlagDet score of 0.610, a recall of 0.820, and a precision of 0.580. This shows how strong embedding models are when used directly on this task. The overview also reports lower performance on the older PAN 2012 dataset, because that dataset focuses on sentence-level plagiarism instead of paragraph-level. These baselines give a useful reference point for evaluating the retrieval approach of our AI microservice, and they confirm that dense embedding retrieval is the strongest general strategy for this task.

Table 2: literature study for Generative Plagiarism Detection

Paper	Approach	Embedding Model	Key Tools	Results	Key Strength	Key Limitation
Mo et al. [1]	Multi-feature fusion (semantic + lexical), graph-based merging, soft-NMS boundary refinement	GloVe (static word embeddings)	spaCy, WordNet, NLTK	PlagDet: 0.584 Recall: 0.727 Precision: 0.487	Winner at PAN 2025; optimal granularity (1.0)	Reduced precision (0.487); preset thresholds need tuning
Su et al. [2]	Hierarchical multi-model ensemble	SBERT, MPNet (transformer);	spaCy, FAISS	PlagDet: 0.496 Recall:	BERT fallback improves	Recall bounded by FAISS top-k;

Paper	Approach	Embedding Model	Key Tools	Results	Key Strength	Key Limitation
	(SBERT + MPNet + TF-IDF), BERT pairwise fallback, block merging	TF-IDF (statistical)		0.578 Precision: 0.600	precision; strong multi-model ensemble	speed-recall trade-off
Luo et al. [3]	Two-stage: TF-IDF cosine + Jaccard filtering, then fine-tuned BERT classifier	TF-IDF (statistical); BERT (fine-tuned classifier)	NLTK	Recall: 0.177 Precision: 0.582	High computational efficiency; reduced search space	Very low recall (0.177); lexical filtering discards semantic matches
Tang et al. [4]	Document-level filtering, sentence embedding + FAISS retrieval, sliding window chunking	intfloat/e5-base-v2 (sentence transformer)	FAISS	Recall: 0.73 Precision: 0.700	High recall via sliding window + e5 embeddings	Precision drops on test set (0.33); threshold overfitting
Greiner-Petter et al. [5]	Official baselines: semantic similarity (decoder LLMs) + lexical n-gram (PAN 2012)	Linq-Embed-Mistral, Qwen, Llama (decoder LLMs)	—	PlagDet: 0.610 Recall: 0.820 (Linq)	Linq baseline outperforms most team submissions; establishes strong upper bound	Performance drop on PAN 2012 (sentence-level dataset)

Across the reviewed approaches, two consistent findings emerge:

- Dense semantic retrieval with transformer embeddings gets much higher recall than methods that only look at shared words. This matters because generative plagiarism, by definition, rewrites the text so it shares almost no exact words with its sources. The low recall (0.177) of Luo et al. [3], who filtered by shared words first, confirms this.

- Precision is the harder problem. Even the best teams find it hard to avoid false positives. The most effective fix in the literature is a second checking step, either a BERT classifier [2] or boundary refinement [1]. Our AI microservice builds on these findings. It uses e5-base-v2 dense embeddings as a baseline [4] and adds BM25 keyword retrieval in a hybrid fusion, so it captures both meaning and shared words. It

also uses the same sliding window chunking as [4]. We then added and tested a cross-encoder reranking stage, described in Chapter 6. It makes the ranking more stable when its score is blended with the hybrid fusion score, instead of being used alone.

3.3.2 PAN 2026 Generative Plagiarism Detection: Comparison of Team Solutions

This section reviews the PAN 2026 Generative Plagiarism Detection shared task, now that the results are out. It looks at the solutions of three other teams that published their notebooks and compares them with our own. The task is the same for everyone: find the source documents that a suspicious document copies from. Because of this, the teams made similar design choices, and the differences between their systems and ours turned out to be small. For each team, we describe their approach and then point out where it matches ours and where it differs.

Bedratyuk and Bedratiuk (2026) — MiniLM Dense Retrieval with Local Re-ranking [9]

Bedratiuk and Bedratiuk built a two-stage system. The first stage uses a small MiniLM Sentence-BERT model to turn whole documents into vectors and finds candidate sources with cosine similarity. The second stage re-ranks the top candidates with three local signals: fragment matching (sentence-level comparison with a coverage check), semantic attention (a softer version of the same idea), and an experimental score that compares normalized groups of sentence vectors. The final score is a weighted sum, with dense retrieval getting most of the weight (0.80). Their main finding is that the re-ranking steps cannot recover a source that the first stage missed, so the first stage is the bottleneck.

Wang et al. (2026) — Query-Chunk Provenance Modeling with Segmented BM25 [7]

Wang et al. built a lightweight system that uses only BM25, with no GPU and no pre-trained models. Their idea is that a generated document mixes content from several sources, so one full-document query becomes noisy and weakens the signal of each source. Instead, they split the suspicious document into paragraph-level chunks of 800 to 3,000 characters. Each chunk searches the corpus on its own with BM25, and common words are filtered out to keep the search fast. The chunk results are then combined with coverage-weighted voting. A source that is matched by many different chunks gets a bonus, because that points to real multi-paragraph reuse and not a single lucky match.

Cody et al. (2026) — Span-Based Hybrid Retrieval with BM25 and GTE [6]

Cody et al. noticed that a generated document mixes several sources, so using the whole document as one query hides the signal of each source. They split the suspicious document into single sentences and search for each sentence separately, in two ways. BM25 searches the full source documents for exact word matches. A dense GTE-base model searches source chunks of 220 to 320 words that follow the paper's section structure, and the best chunk score becomes the document score. The two rankings for each sentence are merged with Reciprocal Rank Fusion, which uses rank positions and not raw scores. CombSUM then adds up the sentence scores into one final ranking of source documents. The system needs no training. Since no development data was released, the team made 200 synthetic queries with an LLM and used them to tune the settings.

Table 3: PAN26 Shared task teams solutions comparison

Team	Approach	Embedding model	Granularity	Fusion/scoring
JLP [6]	Hybrid retrieval (BM25 + GTE)	GTE-base-v1.5 (bi-encoder)	Query: sentence-level Source: structure-aware chunks	RRF: sparse + dense CombSUM: sentence-level to document-level
FousAI [7]	BM25 Segmented	–	Query: paragraph-level	Coverage-weighted voting mechanism*

ENGsTF [8]	Hybrid two stage (BGE-M3 + BM25) + Reranking	BGE-M3 (bi-encoder) + BGE-reranker-v2-m3	Query: full document Sources: overlapped token-based chunks	Normalized weighted linear fusion (α=0.57) Reranker blend (β=0.7)
Bedratiuk-Lab [9]	Two stage: dense retrieval + Reranking(3 signals)	MiniLM(SBERT-style)	Retrieval stage: document-level Reranking stage: sentence-level	Weighted linear fusion**
PAN Baseline	BM25	–	Whole documents	–
PAN Baseline	LINQ Embeddings	Linq-Embed-Mistral		
PAN Baseline	Numeral Overlap	–		

Table 4: PAN26 Shared task results

Team	nDCG @10	nDCG@ 100	R@10	R@100	MRR	Key notes
JLP	**0.799**	**0.833**	**0.854**	**0.963**	**0.976**	Generated a synthetic dataset that helped tuning parameters
FousAI	0.626	0.626	0.630	0.630	0.935	Computationally lightweight
ENGsTF	0.642	0.687	0.671	0.840	0.908	Hyperparameters needed proper tuning
Bedratiuk-Lab	0.385	0.385	0.386	0.386	0.619	Dense retrieval proved to be the strongest component; first stage is a bottleneck (might miss true sources; second stage can't recover)
PAN Baseline	0.553	–	–	–	0.832	
PAN Baseline	0.391	–	0.445	0.726	0.689	Dense only approach with a very large model is not enough, how the query is handled matters
PAN Baseline	0.184	–	0.170	0.220	0.321	

Main findings from the PAN 2026 teams:

- Hybrid retrieval worked best. The two top systems (JLP and ours) both combined BM25 with a dense model. Each method catches what the other misses: BM25 finds exact terms, and dense models find paraphrased meaning.

- Splitting the suspicious document helped. A generated document mixes several sources, so one long query blurs them. JLP (sentences) and FousAI (paragraphs) both searched with smaller pieces and combined the results.

- BM25 is still a strong, cheap baseline. FousAI used only BM25 with no GPU or model.

- A small dense model alone was not enough. Bedratiuk-Lab used MiniLM alone for first-stage retrieval and fell below BM25. Better re-ranking cannot fix a first stage that misses the source.

JLP compared with our system:

Table 5: winning team in PAN26 VS us

	JLP (winner)	Ours
Query handling	Splits the document into sentences and searches each one	Uses the full document as one query
Dense model	GTE-base-v1.5	BGE-M3
Source chunks	Chunks that follow the paper's section structure	Overlapping token-based chunks
Fusion	RRF (rank-based), then CombSUM to add up sentence scores	Weighted linear fusion of scores ($\alpha = 0.57$)
Re-ranking	None	Cross-encoder (BGE-reranker-v2-m3), blended with $\beta = 0.7$
Tuning	200-query synthetic dataset made with an LLM	Hyperparameters tuned by hand

Our system is close to JLP's. Both use a hybrid of BM25 and a dense bi-encoder with chunked source documents, and GTE and BGE-M3 are similar in practice. The main difference is that JLP built a synthetic tuning dataset that helped them choose their settings, while we had to tune ours without one.

3.3.3 AI-Generated Text Detection

This section reviews work on AI-generated text detection, the new component of our system. It first covers the solutions of the highest-ranking teams at the PAN 2024 and PAN 2025 Voight-Kampff tasks, and explains the models and training ideas they used. It then presents a table of important benchmarks and zero-shot baselines for this kind of task, which shows how detectors are usually tested and what they can do without training. Our system works on academic papers and needs to stay lightweight, so we pay close attention to which of these methods could fit those limits.

Table 6: Literature study for AI-Generated Text Detection

Paper	Category	Model(s)	Augmentation	Training Data	Eval / validation data	Generators
[10] mdok	Fine-tuned LLM	Qwen3-14B-Base, QLoRA sequence classification	10% of AI samples with homoglyph attack	PAN25(train + validation)	MIX2k	PAN generators, MIX2k covers 75
[11] Suspiciously Coherent	Ensemble	Qwen3-4B-Base embeddings + Light GBM head (0.47), LoRA Qwen3-14B-Base (0.53)	10% homoglyph attack (as [5]), self-generated obfuscations	PAN25 (train + 95% validation), M4GT subset, self-generated obfuscations	5% PAN25 val, M4GT held-out, RAID, MULTITuDEv2	PAN generators, GPT-4.1-mini, Claude, Gemini, Llama-3.2-3B
[12] DACTYL	Encoder ensemble	Bayesian BERT-tiny ensemble (filters training data), DeBERTa-V3-large, ModernBERT-large, MCGrad calibration	–	DACTYL-complete, LLMtrace, Personagen, MAGA-Bench	Calibration: balanced sample across 13 datasets; test: 13 datasets	GLM 5.1, GPT 5.4, Claude Sonnet 4.6, Claude Haiku 4.5, Mercury 2, MiniMax-M2.5, Qwen3.5-397B-A17B, Kimi-K2.5

[13] Plagiarism-Detector.com	Fine-tuned LLM	Qwen3-14B-Base, QLoRA, focal loss	8+ techniques (humanizer, DIPPER, homoglyphs, style mimicry, eloquent reformat), ~46k docs	PAN25 train + validation + 46k augmented docs	_	PAN generators
[14] DS@GT	Fine-tuned LLM	Qwen2.5-1.5B, LoRA, sliding window + averaged-probabilities	Tested, no gain, dropped	PAN25	PAN25 (validation) HC3, RAID, MAGE, OpenGPTText, AI Detection Pile, M4/SemEval (evaluation)	PAN generators
[15] A&A-detection	Hybrid ensemble	CatBoost (stylometric + model-fingerprint features), DeBERTa-v3-large	_	PAN25	PAN25	PAN generators

Table 7: Results

Paper	PAN26 test	PAN25 test	ELOQUENT Adversarial test	OOD / other eval	Compute	Key limitation
[10] (PAN25 winner)	_	0.989	_	0.696 (MIX2k)	High	Large drop on OOD data
[11]	0.975	0.979	0.892	0.847 (validation)	High	Fixed ensemble weights, still vulnerable to OOD
[12]	0.974	0.949	0.911	0.904 (DetectRL)	High	Complex, costly pipeline, manual hyperparameter tuning
[13]	0.968	0.982	0.936	_	Med	Vulnerable to newer-generation LLMs
[14]	0.921	_	0.771	0.994 (RAID) 0.462 (AI Detection Pile) 0.625 (MAGE)	Low	Highly variable on OOD
[15]	0.858	_	0.683	_	Low	Stylometric cues are weak and easy to avoid

Most top PAN 2024 and 2025 systems follow he same pattern:

- **Fine-tune a strong model.** Most teams fine-tune a pre-trained model for human vs. machine detection. Usually this is a large LLM trained with LoRA or QLoRA.

Sometimes it is an encoder like DeBERTa or ModernBERT.

- **Add something to make it robust.** Teams use data augmentation (homoglyph swaps, paraphrasing, style-mimicking rewrites that try to make AI text look human) or combine several models in an ensemble. Some add calibration or training-data filtering.

- **Test on unseen data.** Many teams hold out out-of-distribution data (new generators or new domains) to check generalization.

- **The main problem is generalization.** Scores are very high on data like the training data (about 0.97 to 0.99) but drop sharply on unseen data. For example, [10] falls from 0.989 to 0.696, and [14] is near perfect on RAID but about 0.45 on AI Detection Pile. The causes are new generators (the newest LLM was the biggest failure for [13]) and new text types.

- **The best systems are costly.** The top results come from large models, so they are expensive to run.

- **No system was tested on academic papers in specific.** PAN training data covers fiction, essays, and news. This is why we look for a lightweight method that works on academic writing.

References

[1] Mo, D., Zhang, H., Zhang, X., & Kong, L. (2025). Using seman c similarity and overlap ra o opmized for generated plagiarism detec on. In CLEF 2025 Working Notes (CEUR Workshop Proceedings, Vol. 4038, pp. 2933–2942). CEUR-WS.org.

[2] Su, Z., Han, Y., Jia, Y., & Kong, L. (2025). Hierarchical genera ve plagiarism detec on method: Notebook for PAN at CLEF 2025. In A. Greiner-Petter, M. Fröbe, J. P. Wahle, T. Ruas, B. Gipp, A. Aizawa, & M. Potthast (Eds.), CEUR Workshop Proceedings: Vol. 4038. Working Notes of CLEF 2025 – Conference and Labs of the Evaluation Forum. CEUR-WS.org.

[3] Luo, J., Huang, M., Liu, B., & Han, Z. (2025). Team jr at Genera ve Plagiarism Detec on 2025: A two-stage approach: From TF-IDF/Jaccard filtering to Transformer classifica on [Notebook for the PAN lab at CLEF 2025]. In A. Greiner-Petter, M. Fröbe, J. P. Wahle, T. Ruas, B. Gipp, A. Aizawa, & M. Potthast (Eds.), CEUR Workshop Proceedings: Vol. 4038. Working Notes of CLEF 2025 – Conference and Labs of the Evaluation Forum. CEUR WS.org.

[4] Tang, J., Hu, Q., & Han, Z. (2025). Efficient plagiarism detec on via sentence embeddings and FAISS-based retrieval. Notebook for the PAN lab at CLEF 2025. Foshan University. [5] Greiner-Pe er, A., Fröbe, M., Wahle, J. P., Ruas, T., Gipp, B., Aizawa, A., & Po hast, M. (2025). Overview of the plagiarism detec on task at PAN 2025. arXiv preprint arXiv:2510.06805. [Also in Proceedings of the 16th Interna onal Conference of the CLEF Associa on (CLEF 2025), CEUR-WS.org, Vol. 4038, Paper 280].

[5] Greiner-Petter, A., Fröbe, M., Wahle, J. P., Ruas, T., Gipp, B., Aizawa, A., & Potthast, M. (2025). *Overview of the plagiarism detection task at PAN 2025*. arXiv. https://doi.org/10.48550/arXiv.2510.06805

[6] Cody, J., et al. (2026). Span-based hybrid source retrieval for generative plagiarism detection. In E. Sánchez Salido et al. (Eds.), *Working Notes of CLEF 2026 – Conference and Labs of the Evaluation Forum* (Vol. 4283, pp. 5439–5449). CEUR-WS.org.

[7] Wang, et al. (2026). Source retrieval via query-chunk provenance modeling for generative plagiarism detection. In E. Sánchez Salido et al. (Eds.), *Working Notes of CLEF 2026 – Conference and Labs of the Evaluation Forum* (Vol. 4283, pp. 5727–5732). CEUR-WS.org.

[8] Alkayyem, F., Alkhateeb, T., Sonbol, R., & Alhossny, R. (2026). Team ENGsTF at PAN 2026: Hybrid sparse-dense retrieval with two-stage score fusion for generative plagiarism detection. In E. Sánchez Salido et al. (Eds.), *Working Notes of CLEF 2026 – Conference and Labs of the Evaluation Forum* (Vol. 4283, pp. 5382–5388). CEUR-WS.org.

[9] Bedratyuk, D., & Bedratiuk, U. (2026). From embedding similarity to semantic canonization for generated plagiarism detection. In E. Sánchez Salido et al. (Eds.), *Working Notes of CLEF 2026 – Conference and Labs of the Evaluation Forum* (Vol. 4283, pp. 5416–5421). CEUR-WS.org.

[10] D. Macko, mdok of KInIT: Robustly Fine-tuned LLM for Binary and Multiclass AI-Generated Text Detection, in: G. Faggioli, N. Ferro, P. Rosso, D. Spina (Eds.), Working Notes of CLEF 2025 - Conference and Labs of the Evaluation Forum, CEUR-WS.org, 2025.

[11] Z. Antoszewska, N. M. Bryła, A. V. Szczerba, Team Suspiciously Coherent at PAN 2026: We stacked two Qwens and hoped for the best, in: E. Sánchez Salido, A. Barrón-Cedeño, A. G. S. de Herrera, S. MacAvaney, J. M. Struß (Eds.), Working Notes of CLEF 2026 - Conference and Labs of the Evaluation Forum, CEUR-WS.org, 2026.

[12] S. Thorat, Team DACTYL at PAN 2026: Bayesian Data Mixing and Empirical X-risk Minimization for AI-text Detection, in: E. Sánchez Salido, A. Barrón-Cedeño, A. G. S. de Herrera, S. MacAvaney, J. M. Struß (Eds.), Working Notes of CLEF 2026 - Conference and Labs of the Evaluation Forum, CEUR-WS.org, 2026.

[13] Y. Palkovskii, A. Belov, Team Plagiarism-Detector.com at PAN 2026: Strong Detector, Moving Target – Adversarial Augmentation and Out-of-Distribution Findings, in: E. Sánchez Salido, A. Barrón-Cedeño, A. G. S. de Herrera, S. MacAvaney, J. M. Struß (Eds.), Working Notes of CLEF 2026 - Conference and Labs of the Evaluation Forum, CEUR-WS.org, 2026.

[14] H. Basava, M. Gustineli, Fine-Tuning Fundamentals and Data Augmentation: DS@GT ARC at Voight-Kampff 2026, in: E. Sánchez Salido, A. Barrón-Cedeño, A. G. S. de Herrera, S. MacAvaney, J. M. Struß (Eds.), Working Notes of CLEF 2026 - Conference and Labs of the Evaluation Forum, CEUR-WS.org, 2026.

[15] A. Voulgaraki, A. Zamyshevskaya, Team A&A-detection at PAN 2026: A Hybrid Ensemble of Stylometric, Perplexity, and Neural Features for Voight-Kampff Generative AI Detection, in: E. Sánchez Salido, A. Barrón-Cedeño, A. G. S. de Herrera, S. MacAvaney, J. M. Struß (Eds.), Working Notes of CLEF 2026 - Conference and Labs of the Evaluation Forum, CEUR-WS.org, 2026.